How-To Library

Informatica

Dynamic Data Masking Default Masking Functionality

© Copyright Informatica LLC 2018, 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

In Dynamic Data Masking™ version 9.8.4, the default masking functionality is enhanced in a way that changes its behavior. This H2L provides examples of different types of SQL statements to show the differences between the old and new default masking functionalities.

Supported Versions

- Dynamic Data Masking 9.8.4 or later

Table of Contents

Overview. . . . 3
New Masking Functionality Enhancements. . . . 3
General Use Cases. . . . 4
Use Case Descriptions. . . . 5
General Use Case #1: SQL statement with column list as asterisk (*) (implicit column list) 5
General Use Case #2: SQL statement with explicit column list (that does not contain the WHERE, GROUP BY, HAVING, or ORDER BY clauses). . . . 6
General Use Case #3: SQL statement that contains the WHERE, GROUP BY, HAVING, and ORDER BY clauses. . . . 6
General Use Case #4: SQL statement that contains the WHERE, GROUP BY, HAVING, and ORDER BY clauses, as well as UNION. . . . 8
General Use Case #5: SQL statement that contains a subquery. . . . 10
General Use Case #6: SQL Statement that contains a subquery with the UNION operation. . . . 12
General Use Case #7: SQL statement that contains a subquery containing GROUP BY, HAVING, and ORDER BY clauses. . . . 14
General Use Case #8: SQL Statement that contains a DISTINCT keyword. . . . 16
General Use Case #9: SQL statements having subquery without alias. . . . 17
General Use Case #10: SQL statements where candidate column is part of a expression without column alias. . . . 18
General Use Case #11: SQL statement contains a WITH clause. . . . 19
Microsoft SQL Server and Sybase Use Cases. . . . 20
Use Case #1: SQL batch (several SQL statements or commands) that do not contain set statements or the USE <Database> command. . . . 20
Use Case #2: SQL blocks that contain the USE <Database> command. . . . 21
Use Case #3: SQL blocks that contain the SET QUOTED IDENTIFIER command. . . . 21
Use Case #4: SQL blocks that contain the EXECUTE AS command. . . . 22
Use Cases Specific to Security Rule Design. . . . 24
Security Rule Design Use Case #1. . . . 24

Security Rule Design Use Case #2. 25

Overview

In Dynamic Data Masking version 9.8.4, the default masking functionality is a new functionality that enhances the previous masking functionality in multiple ways. To apply masking, the new default masking functionality modifies some SQL statements in a different way than in previous versions, which used the old masking functionality by default.

The following table shows the differences between versions and masking functionalities:

Release Version	Old Masking Functionality	New Masking Functionality
DDM 9.8.3 and previous	Default.	Enabled by setting the JVM property -Drules.mask.new=true in the `jvm.params` file
DDM 9.8.4 +	Enabled by setting the JVM property -Drules.mask.new=false in the `jvm.params` file.	Default.

This H2L article provides examples of SQL statements and a comparison of how the statement was modified in previous versions versus how it is modified by default in version 9.8.4. For each use case, a description of the old and new masking functionality is given.

New Masking Functionality Enhancements

The new masking functionality, introduced in version 9.8.4 as the default functionality, has the following advantages over the old functionality.

- The new functionality relies on database case sensitivity while the old masking functionality uses double quotes and uppercase characters for table aliases.
- With the new masking functionality for SELECT queries, while replacing asterisks with the corresponding column list, you can use the AUTH_QUOTED_IDENTIFIER symbol to decide if the identifiers should be with double quotes or not in the masked query.
- The new masking functionality uses a lazy masking algorithm. This means that when a column which is candidate for masking in the inner subquery, the following rules apply:
  - If the candidate column is not part of the expression, the corresponding column in the outer query is masked.
  - If the candidate column is part of the expression, the candidate column is masked in the inner subquery.

General Use Cases

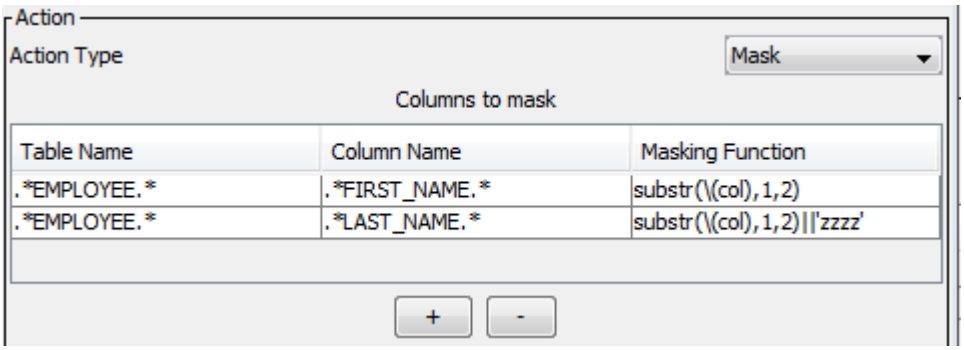
This H2L article contains descriptions of common use cases that demonstrate the old and new functionalities.

The following table summarizes the use cases:

#	Use Case	Masking Functionality
1	SQL statement with the column list as asterisk (*) (implicit column list).	Both the old and new masking approaches are applicable. For more details, see section Use Case 1.
2	SQL statement with an explicit column list that does not contain the WHERE, GROUP BY, HAVING, or ORDER BY clauses.	Both the old and new masking approaches are applicable. For more details, see section Use Case 2.
3	SQL statement with an explicit column list containing the WHERE, GROUP BY, HAVING, or ORDER BY clauses	Both the old and new masking approaches are applicable. For more details, see section Use Case 3.
4	SQL statement with an explicit column list containing the WHERE, GROUP BY, HAVING, or ORDER BY clauses together with UNION.	Both the old and new masking approaches are applicable. For "Keep Original Number of Rows"= Y only. For more details, see section Use Case 4.
5	SQL statement contains a subquery.	The new masking approach is applicable. For more details, see section Use Case 5.
6	SQL statement contains a subquery with the UNION operation.	The new masking approach is applicable. For more details, see section Use Case 6.
7	SQL statement contains a subquery containing GROUP BY, HAVING, and ORDER BY clauses.	The new masking approach is applicable. For more details, see section Use Case 7.
8	SQL statement contains a DISTINCT keyword.	The new masking approach is applicable. For more details, see section Use Case 8.
9	SQL statements contains a subquery without an alias.	The new masking approach is applicable. For more details, see section Use Case 9.
10	SQL statements where the candidate column is part of an expression without a column alias.	The new masking approach is applicable. For more details, see section Use Case 10.
11	SQL statement contains a WITH clause.	The new masking approach is applicable. For more details, see section Use Case 11.

Use Case Descriptions

You define a security rule on the EMPLOYEES table as shown in the following image:



General Use Case #1: SQL statement with column list as asterisk (*) (implicit column list)

Use case #1 provides an example for a SQL statement that contains an asterisk (*). In this scenario the original Number of Rows check box is not relevant.

For both the old and new default masking functionality, Dynamic Data Masking expands the SQL statement to replace the asterisk with the list of the column names and then applies masking.

Example statement:

```
SELECT * from EMPLOYEES
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification
N or Y	SELECT "EMPLOYEES" . "EMPLOYEE_ID" , substr("EMPLOYEES"."FIRST_NAME", 1,2) "FIRST_NAME" , substr("EMPLOYEES"."LAST_NAME", 1,2)||'zzzz' "LAST_NAME" , "EMPLOYEES" . "EMAIL" , "EMPLOYEES" . "PHONE_NUMBER" , "EMPLOYEES" . "HIRE_DATE" , "EMPLOYEES" . "JOB_ID" , "EMPLOYEES" . "SALARY" , "EMPLOYEES" . "COMMISSION_PCT" , "EMPLOYEES" . "MANAGER_ID" , "EMPLOYEES" . "DEPARTMENT_ID" from EMPLOYEES	SELECT EMPLOYEES . "EMPLOYEE_ID" ,substr(EMPLOYEES . "FIRST_NAME" ,1,2) "FIRST_NAME" ,substr(EMPLOYEES . "LAST_NAME" ,1,2)||'zzzz' "LAST_NAME" , EMPLOYEES . "EMAIL" , EMPLOYEES . "PHONE_NUMBER" , EMPLOYEES . "HIRE_DATE" , EMPLOYEES . "JOB_ID" , EMPLOYEES . "SALARY" , EMPLOYEES . "COMMISSION_PCT", EMPLOYEES . "MANAGER_ID" , EMPLOYEES . "DEPARTMENT_ID" from EMPLOYEES

The following table describes the example masking scenarios when the Dynamic Data Masking symbol AUTH_QUOTED_IDENTIFIER is set to OFF. This applies for the new masking functionality only.

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification
N or Y	SELECT "EMPLOYEES" . "EMPLOYEE_ID" , substr("EMPLOYEES"."FIRST_NAME", 1,2) "FIRST_NAME" , substr("EMPLOYEES"."LAST_NAME", 1,2)||'zzzz' "LAST_NAME" , "EMPLOYEES" . "EMAIL" , "EMPLOYEES" . "PHONE_NUMBER" , "EMPLOYEES" . "HIRE_DATE" , "EMPLOYEES" . "JOB_ID" , "EMPLOYEES" . "SALARY" , "EMPLOYEES" . "COMMISSION_PCT" , "EMPLOYEES" . "MANAGER_ID" , "EMPLOYEES" . "DEPARTMENT_ID"from EMPLOYEES	SELECT EMPLOYEES . EMPLOYEE_ID , substr(EMPLOYEES . FIRST_NAME,1,2) FIRST_NAME , substr(EMPLOYEES . LAST_NAME,1,2)||'zzzz' LAST_NAME , EMPLOYEES . EMAIL , EMPLOYEES. PHONE_NUMBER , EMPLOYEES . HIRE_DATE , EMPLOYEES . JOB_ID, EMPLOYEES . SALARY , EMPLOYEES . COMMISSION_PCT , EMPLOYEES . MANAGER_ID , EMPLOYEES . DEPARTMENT_IDfrom EMPLOYEES

General Use Case #2: SQL statement with explicit column list (that does not contain the WHERE, GROUP BY, HAVING, or ORDER BY clauses)

Use case #2 provides an example of a SQL statement that does not contain the WHERE, GROUP BY, HAVING, or ORDER BY clauses. In this scenario, the Original Number of Rows check box is not relevant.

Example statement:

```
select FIRST_NAME,LAST_NAME||'.' from EMPLOYEES
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N or Y	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' from EMPLOYEES	select substr(FIRST_NAME,1,2) FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' C_X0 from EMPLOYEES	Old masking functionality: Col List – Y New masking functionality: Col List – Y If a candidate column is a part of an expression and does not have a column alias, the new masking algorithm adds a column alias to the candidate column. This is done to hide the masking function if applied.

General Use Case #3: SQL statement that contains the WHERE, GROUP BY, HAVING, and ORDER BY clauses

Use case #3 provides an example of a SQL statement that contains the WHERE, GROUP BY, HAVING, and ORDER BY clauses.

Example statement:

```
select FIRST_NAME, LAST_NAME||'.', DEPARTMENT_ID, COUNT(DEPARTMENT_ID) from EMPLOYEES WHERE
LAST_NAME LIKE '%a%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME, LAST_NAME,DEPARTMENT_ID
HAVING COUNT(DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY
FIRST_NAME,LAST_NAME,DEPARTMENT_ID
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.', DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from EMPLOYEES WHERE LAST_NAME LIKE '%a%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME ",1,2)||'zzzz' , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 ANDsubstr("LAST_NAME" ,1,2)||'zzzz'!= 'Brown' ORDER BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME ",1,2)||'zzzz' , DEPARTMENT_ID	select substr(FIRST_NAME,1,2) FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' C_X3 , DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from EMPLOYEES WHERE LAST_NAME LIKE '%a%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY substr(FIRST_NAME , 1,2) , substr(LAST_NAME , 1,2)||'zzzz', DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID)>= 1 AND substr(LAST_NAME, 1,2)||'zzzz' != 'Brown' ORDER BY substr(FIRST_NAME , 1,2) ,substr(LAST_NAME , 1,2)||'zzzz', DEPARTMENT_ID	Old and new masking functionality: - Col List - Y - Where – N - Group By – Y - Having – Y - Order By – Y
Y	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.', DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from EMPLOYEES WHERE LAST_NAME LIKE '%a%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME, LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID	select substr(FIRST_NAME,1,2) FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' C_X0 , DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from EMPLOYEES WHERE LAST_NAME LIKE '%a%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID)>= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID	Old and new masking functionality: - Col List - Y - Where – N - Group By – N - Having – N - Order By – N

General Use Case #4: SQL statement that contains the WHERE, GROUP BY, HAVING, and ORDER BY clauses, as well as UNION

Use case #4 provides an example of a SQL statement that contains the WHERE, GROUP BY, HAVING, and ORDER BY clauses, as well as UNION.

Example statement:

```
select FIRST_NAME,LAST_NAME||'.' LAST_NAME,DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S
%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME, LAST_NAME,DEPARTMENT_ID HAVING LAST_NAME !=
'SMITH' UNION select FIRST_NAME,LAST_NAME||'.' LAST_NAME,DEPARTMENT_ID from EMPLOYEES WHERE
LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME,LAST_NAME,DEPARTMENT_ID
HAVING LAST_NAME != 'SMITH' ORDER BY FIRST_NAME, LAST_NAME,DEPARTMENT_ID
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID HAVING substr("LAST_NAME", 1,2)||'zzzz' != 'SMITH' UNION select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID HAVING substr("LAST_NAME", 1,2)||'zzzz' != 'SMITH' ORDER BY substr("FIRST_NAME", 1,2),substr("LAST_NAME", 1,2)||'zzzz' , DEPARTMENT_ID	select substr(FIRST_NAME,1,2) FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' LAST_NAME , DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY substr(FIRST_NAME , 1,2) , substr(LAST_NAME , 1,2)||'zzzz', DEPARTMENT_ID HAVING substr(LAST_NAME , 1,2)||'zzzz'!= 'SMITH' UNION select substr(FIRST_NAME,1,2) FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' LAST_NAME , DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY substr(FIRST_NAME , 1,2) , substr(LAST_NAME , 1,2)||'zzzz', DEPARTMENT_ID HAVING substr(LAST_NAME , 1,2)||'zzzz'!= 'SMITH' ORDER BY substr(FIRST_NAME , 1,2),substr(LAST_NAME, 1,2)||'zzzz' , DEPARTMENT_ID	In both the old and new functionality, masking is applied to each query in the complete UNION statements individually. In both the old and new masking functionality, when Keep Original Number of rows is unchecked: Masked statements created by Dynamic Data Masking are not valid and throw an exception, because masking is applied to the column aliases in the ORDER BY clause.
Y	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING LAST_NAME != 'SMITH' UNION select substr("FIRST_NAME",	select substr(FIRST_NAME,1,2) FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' LAST_NAME , DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING LAST_NAME != 'SMITH' UNION select substr(FIRST_NAME,1,2) FIRST_NAME ,substr(LAS	Masked as expected. Each statement is handled individually. Old and new masking functionality: - Col List - Y - Where – N - Group By – N - Having – N - Order By – N

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
	1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz'||'.' LAST_NAME , DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING LAST_NAME != 'SMITH' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID	T_NAME ,1,2)||'zzzz'||'.' LAST_NAME , DEPARTMENT_ID from EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING LAST_NAME != 'SMITH' ORDER BY FIRST_NAME , LAST_NAME, DEPARTMENT_ID	

General Use Case #5: SQL statement that contains a subquery

Use case #5 provides an example of a SQL statement that contains a subquery.

Example statement:

```
select FIRST_NAME,LAST_NAME,DEPARTMENT_ID, COUNT(DEPARTMENT_ID) from (SELECT
FIRST_NAME,LAST_NAME||'.' LAST_NAME, DEPARTMENT_ID FROM EMPLOYEES) A WHERE LAST_NAME LIKE '%S%'
OR FIRST_NAME LIKE '%John%' GROUP BY FIRST_NAME,LAST_NAME,DEPARTMENT_ID HAVING
COUNT(DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME,LAST_NAME,DEPARTMENT_ID
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' "LAST_NAME" , DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from (SELECT substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES) A WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%John%' GROUP BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND substr("LAST_NAME", 1,2)||'zzzz' != 'Brown' ORDER BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID	select substr(FIRST_NAME,1,2) FIRST_NAME , LAST_NAME LAST_NAME , DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from (SELECT FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES) A WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%John%' GROUP BY substr(FIRST_NAME , 1,2) , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY substr(FIRST_NAME , 1,2) , LAST_NAME , DEPARTMENT_ID	The old masking functionality resulted in incorrect masking, because columns are masked twice in both OUTER and INNER queries. In the new masking functionality, Dynamic Data Masking masks the candidate column inside the subquery if it is part of an expression. Otherwise, Dynamic Data Masking masks the candidate column in the OUTER query.
Y	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' "LAST_NAME" , DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from (SELECT substr("FIRST_NAME", 1,2)"FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES) A WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%John%' GROUP BY FIRST_NAME , LAST_NAME ,	select substr(FIRST_NAME,1,2) FIRST_NAME , LAST_NAME LAST_NAME , DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from (SELECT FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES) A WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%John%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID)>= 1	The old masking functionality resulted in incorrect masking, because columns are masked twice in both OUTER and INNER queries. In the new masking functionality, Dynamic Data Masking masks the candidate column inside the subquery if it is part of an expression. Otherwise, Dynamic Data Masking masks the candidate column in the OUTER query.

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
	DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID	AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID	

General Use Case #6: SQL Statement that contains a subquery with the UNION operation

Use case #6 provides an example of a SQL Statement that contains a subquery with the UNION operation.

Example statement:

```
select FIRST_NAME,LAST_NAME,DEPARTMENT_ID, COUNT(DEPARTMENT_ID) from (SELECT
FIRST_NAME,LAST_NAME||'.' LAST_NAME, DEPARTMENT_ID FROM EMPLOYEES UNION SELECT
FIRST_NAME,LAST_NAME LAST_NAME, DEPARTMENT_ID FROM EMPLOYEES UNION SELECT DEPARTMENT_NAME
FIRST_NAME, DEPARTMENT_NAME LAST_NAME, DEPARTMENT_ID FROM DEPARTMENTS) A WHERE LAST_NAME LIKE
'%S%' OR FIRST_NAME LIKE '%JOHN%'GROUP BY FIRST_NAME,LAST_NAME,DEPARTMENT_ID HAVING
COUNT(DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME,LAST_NAME,DEPARTMENT_ID
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' "LAST_NAME" , DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from (SELECT substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES UNION SELECT substr("FIRST_NAME", 1,2) "FIRST_NAME",substr("LAST_NAME",1,2)||'zzzz' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES UNION SELECT DEPARTMENT_NAME FIRST_NAME ,DEPARTMENT_NAME LAST_NAME, DEPARTMENT_ID FROM DEPARTMENTS) A WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND substr("LAST_NAME", 1,2)||'zzzz' != 'Brown' ORDER BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID	select FIRST_NAME , LAST_NAME LAST_NAME , DEPARTMENT_ID , COUNT (DEPARTMENT_ID)from (SELECT substr(FIRST_NAME ,1,2) FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz'|| '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES UNION SELECT substr(FIRST_NAME,1,2) FIRST_NAME ,substr(LAST_NAME ,1,2)||'zzzz' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES UNION SELECT DEPARTMENT_NAME FIRST_NAME , DEPARTMENT_NAME LAST_NAME , DEPARTMENT_ID FROM DEPARTMENTS) A WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%'GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME, DEPARTMENT_ID	The old masking functionality resulted in incorrect masking, because columns are masked twice, both in the outer and inner queries. New masking functionality: When a subquery contains UNION, the decision to mask the candidate columns in the inner or outer query is Dynamic Data Masking analyzes all of the subqueries associated with the UNION operation. For each subquery associated with the UNION operation, Dynamic Data Masking conducts the following analysis: Candidate column masked in the INNER subquery: - The candidate column is masked in the inner query if it is part of an expression. - The candidate column is not part of an expression, but the corresponding column in the other subquery or subqueries (that are part of UNION) is part of an expression and a candidate for masking. - An inner subquery that contains the candidate column is only masked when it is not part of an expression. The corresponding column in the other subquery or subqueries (which are part of the UNION) is not a candidate for masking. Candidate column masked in the outer query: In all other cases, the candidate column is masked in the outer query.
Y	select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' "LAST_NAME" , DEPARTMENT_ID , COUNT	select substr(FIRST_NAME,1,2) FIRST_NAME , LAST_NAME LAST_NAME , DEPARTMENT_ID , COUNT (DEPARTMENT_ID) from	The old masking functionality resulted in incorrect masking, because columns are masked twice, both in the outer and inner queries. New masking functionality:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
	(DEPARTMENT_ID) from (SELECT substr("FIRST_NAME", 1,2)"FIRST_N AME" ,substr("LAST_NAM E",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES) A WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%John%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID	(SELECT FIRST_NAME ,substr(LAS T_NAME ,1,2)||'zzzz'|| '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES) A WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%John%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID)>= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID	When a subquery contains UNION, the decision to mask the candidate columns in the inner or outer query is Dynamic Data Masking analyzes all of the subqueries associated with the UNION operation. For each subquery associated with the UNION operation, Dynamic Data Masking conducts the following analysis: Candidate column masked in the INNER subquery: - The candidate column is masked in the inner query if it is part of an expression. - The candidate column is not part of an expression, but the corresponding column in the other subquery or subqueries (that are part of UNION) is part of an expression and a candidate for masking. - An inner subquery that contains the candidate column is only masked when it is not part of an expression. The corresponding column in the other subquery or subqueries (which are part of the UNION) is not a candidate for masking. Candidate column masked in the outer query: In all other cases, the candidate column is masked in the outer query.

General Use Case #7: SQL statement that contains a subquery containing GROUP BY, HAVING, and ORDER BY clauses

Use case #7 provides an example of a SQL statement that contains a subquery containing GROUP BY, HAVING, and ORDER BY clauses.

Example statement:

```
select FIRST_NAME,LAST_NAME,DEPARTMENT_ID from (SELECT FIRST_NAME,LAST_NAME||'.' LAST_NAME,
DEPARTMENT_ID FROM EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY
FIRST_NAME,LAST_NAME,DEPARTMENT_ID HAVING COUNT(DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown'
ORDER BY FIRST_NAME,LAST_NAME,DEPARTMENT_ID) A GROUP BY first_name,last_name,DEPARTMENT_ID
Having COUNT(DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY
FIRST_NAME,LAST_NAME,DEPARTMENT_ID
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N	select substr("FIRST_NAME", 1,2) "FIRST_NAME" , substr("LAST_NAME", 1,2)||'zzzz' "LAST_NAME" , DEPARTMENT_ID from (SELECT substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND substr("LAST_NAME", 1,2)||'zzzz' != 'Brown' ORDER BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID) A GROUP BY substr("FIRST_NAME", 1,2) ,substr("LAST_NAME",1,2)||'zzzz' , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND substr("LAST_NAME", 1,2)||'zzzz' != 'Brown' ORDER BY substr("FIRST_NAME", 1,2) , substr("LAST_NAME", 1,2)||'zzzz' , DEPARTMENT_ID	select substr(FIRST_NAME ,1,2) FIRST_NAME , LAST_NAME LAST_NAME , DEPARTMENT_ID from (SELECT FIRST_NAME , substr(LAST_NAME , 1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID)>= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID) A GROUP BY substr(first_name ,1,2) , last_name , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY substr(FIRST_NAME , 1,2) , LAST_NAME , DEPARTMENT_ID	The old masking functionality resulted in incorrect masking, because columns are masked twice, in both the outer and inner queries. Inner Query (for all the columns): - Group By - Y - Having - Y - Order By – Y Outer Query (only for columns masked in the outer query): - Group By - Y - Having - Y - Order By – Y New masking functionality: If candidate columns are not masked in the inner query, then they are masked in the outer query, as per the following: Inner Query (for all the columns): - Group By - N - Having - N - Order By – N Outer Query (only for columns masked in the outer query): - Group By - Y - Having - Y - Order By – Y
Y	select substr("FIRST_NAME", 1,2) "FIRST_NAME" , substr("LAST_NAME", 1,2)||'zzzz' "LAST_NAME" , DEPARTMENT_ID from (SELECT	select substr(FIRST_NAME ,1,2) FIRST_NAME , LAST_NAME LAST_NAME , DEPARTMENT_ID from (SELECT FIRST_NAME , substr(LAST_NAME ,	The old masking functionality resulted in incorrect masking, because columns are masked twice, in both the outer and inner queries. Inner Query (for all the columns): - Group By - N - Having - N

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
	substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID) A GROUP BY first_name , last_name , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID	1,2)||'zzzz' || '.' LAST_NAME , DEPARTMENT_ID FROM EMPLOYEES WHERE LAST_NAME LIKE '%S%' OR FIRST_NAME LIKE '%JOHN%' GROUP BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID) >= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME , LAST_NAME , DEPARTMENT_ID) A GROUP BY first_name , last_name , DEPARTMENT_ID HAVING COUNT (DEPARTMENT_ID)>= 1 AND LAST_NAME != 'Brown' ORDER BY FIRST_NAME ,LAST_NAME , DEPARTMENT_ID	- Order By – N Outer Query (only for columns masked in the outer query): - Group By - N - Having - N - Order By – N New masking functionality: If candidate columns are not masked in the inner query, then they are masked in the outer query, as per the following: Inner Query (for all the columns): - Group By - N - Having - N - Order By – N Outer Query (only for columns masked in the outer query): - Group By - N - Having - N - Order By – N

General Use Case #8: SQL Statement that contains a DISTINCT keyword

Use case #8 provides an example of a SQL Statement that contains a DISTINCT keyword.

Example statement:

```
SELECT DISTINCT FIRST_NAME,LAST_NAME||'.' FROM EMPLOYEES
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N	SELECT DISTINCT substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' FROM EMPLOYEES	SELECT DISTINCT substr(FIRST_NAME ,1,2) FIRST_NAME , substr(LAST_NAME , 1,2)||'zzzz' || '.' C_X0 FROM EMPLOYEES	Old masking functionality: Masks the query. The Keep Original Number of Rows check box is not relevant in this use case. New masking functionality: When Keep Original Number of Rows is enabled, Dynamic Data Masking moves the original statement to the subquery and applies masking on the resultant query that is formed.
Y	SELECT DISTINCT substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' || '.' FROM EMPLOYEES	SELECT substr(T_X0 . "FIRST_NAME" ,1,2) "FIRST_NAME" , T_X0 . "C_X0" "C_X0" FROM (SELECT DISTINCT FIRST_NAME , substr(LAST_NAME , 1,2)||'zzzz' || '.' C_X0 FROM EMPLOYEES) T_X0	Old masking functionality: Masks the query. The Keep Original Number of Rows check box is not relevant in this use case. New masking functionality: When Keep Original Number of Rows is enabled, Dynamic Data Masking moves the original statement to the subquery and applies masking on the resultant query that is formed.

General Use Case #9: SQL statements having subquery without alias

Use case #9 provides an example of a scenario where Dynamic Data Masking adds a table alias to the original query.

Example statement:

```
select * from (select first_name,last_name,DEPARTMENT_ID,salary from EMPLOYEES)
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N	select T_X0 . "FIRST_NAME" , T_X0 . "LAST_NAME" , T_X0 . "DEPARTMENT_ID" , T_X0 ."SALARY" from (select substr("FIRST_NAME", 1,2) "FIRST_NAME" ,substr("LAST_NAME",1,2)||'zzzz' "LAST_NAME" , DEPARTMENT_ID ,salary from EMPLOYEES) T_X0	select substr(T_X2 . "FIRST_NAME" ,1,2) "FIRST_NAME" , substr(T_X2 . "LAST_NAME" ,1,2)||'zzzz' " LAST_NAME" , T_X2 . "DEPARTMENT_ID" , T_X2 . "SALARY" from (select first_name , last_name , DEPARTMENT_ID , salary from EMPLOYEES) T_X2	Dynamic Data Masking adds a table alias when the original statement does not contain an alias for the inner subquery and the outer query has an asterisk.
Y	select T_X0 . "FIRST_NAME" , T_X0 . "LAST_NAME" , T_X0 . "DEPARTMENT_ID" , T_X0 ."SALARY" from (select substr("FIRST_NAME", 1,2) "FIRST_NAME" , substr("LAST_NAME", 1,2)||'zzzz' "LAST_NAME" , DEPARTMENT_ID , salary from EMPLOYEES) T_X0	select substr(T_X2 . "FIRST_NAME" ,1,2) "FIRST_NAME" , substr(T_X2 . "LAST_NAME" ,1,2)||'zzzz' " LAST_NAME" , T_X2 . "DEPARTMENT_ID" , T_X2 ."SALARY" from (select first_name , last_name , DEPARTMENT_ID , salary from EMPLOYEES) T_X2	Dynamic Data Masking adds a table alias when the original statement does not contain an alias for the inner subquery and the outer query has an asterisk.

General Use Case #10: SQL statements where candidate column is part of a expression without column alias

Use case #10 provides an example of a scenario where Dynamic Data Masking adds a column alias to the original query.

Example statement:

```
SELECT DEPARTMENT_ID, first_name||'_NAME' ,salary FROM EMPLOYEES
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
Y or N	SELECT DEPARTMENT_ID ,substr("FIRST_NAME",1,2) || '_NAME' , salary FROM EMPLOYEES	SELECT DEPARTMENT_ID , substr(first_name ,1,2) || '_NAME' C_X3 , salary FROM EMPLOYEES	Old masking functionality: Dynamic Data Masking masks the query as expected. New masking functionality: If the table column is a candidate for masking, is a part of an expression, and doesn't have a column alias, then the masking function is applied on the candidate column and a column alias is dynamically added to the corresponding column in the SQL query. This is done to avoid showing the masking function in the column header on the client side.

General Use Case #11: SQL statement contains a WITH clause

Use case #11 provides an example of a scenario where the SQL statement contains a WITH clause.

Example statement:

```
WITH EMP1 AS (SELECT FIRST_NAME,LAST_NAME FROM EMPLOYEES)SELECT E.FIRST_NAME, E.LAST_NAME||'.'
FROM EMPLOYEES E, EMP1 E1 WHERE E.FIRST_NAME = E1.FIRST_NAME AND E.LAST_NAME = E1.LAST_NAME
```

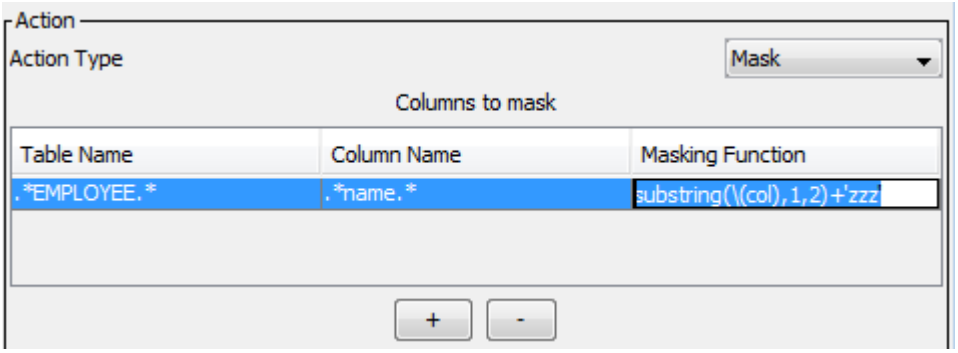
The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N or Y	WITH EMP1 AS (SELECT substr("FIRST_NAME", 1,2) "FIRST_NAME" , substr("LAST_NAME", 1,2)||'zzzz' "LAST_NAME" FROM EMPLOYEES) SELECT substr("E"."FIRST_NAME", 1,2) "FIRST_NAME" , substr("E"."LAST_NAME", 1,2)||'zzzz'|| '.' FROM EMPLOYEES E , EMP1 E1 WHERE E . FIRST_NAME = E1 .FIRST_NAME AND E . LAST_NAME = E1 . LAST_NAME	SELECT substr(E . FIRST_NAME ,1,2) FIRST_NAME , substr(E . LAST_NAME ,1,2)||'zzzz' || '.' C_X1 FROM EMPLOYEES E , (**SELECT FIRST_NAME , LAST_NAME FROM EMPLOYEES**) E1 WHERE E . FIRST_NAME = E1 . FIRST_NAME AND E . LAST_NAME = E1 . LAST_NAME	The Keep Original Number of Rows check box is not relevant since the statement does not contain GROUP BY, HAVING and ORDER BY clauses. The old masking functionality resulted in incorrect masking, because it masked in both the WITH clause and the associated SQL statement. New masking functionality: Before applying masking, Dynamic Data Masking rewrites the WITH clause statement to a statement that has a subquery.

Microsoft SQL Server and Sybase Use Cases

This section describes use cases specific to Microsoft SQL Server and Sybase databases, with SET QUOTED_IDENTIFIER, EXECUTE AS and Use <Database> used in the SQL batch.

You define a security rule on the EMPLOYEES table as shown in the following image:



Use Case #1: SQL batch (several SQL statements or commands) that do not contain set statements or the USE <Database> command

Use case #1 provides an example of a scenario with SQL batch (several SQL statements or commands) that do not contain set statements or the USE <Database> command.

Example statement:

```
EXECUTE GetTaskEvents2 'ALFKI',12209
select * from EMPLOYEES;
select * from customers where country ='Mexico'
DECLARE @ret nvarchar(15);
EXEC @ret = dbo.ufnGetInventoryStock @EMPID = 201
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N or Y	EXECUTE GetTaskEvents2 'ALFKI' , 12209 select "EMPLOYEES" . "EMPID" , substring("EMPLOYEES"." EMPNAM E",1,2)+'zzz' "EM PNAME" , "EMPLOYEES" . "SAL" from EMPLOYEES ; select * from customers where country = 'Mexico' DECLARE @ret nvarchar (15) ; EXEC @ret = dbo . ufnGetInventoryStock @EMPID = 201	EXECUTE GetTaskEvents2 'ALFKI' , 12209 select "EMPLOYEES" . "EMPID" ,substring("EMP LOYEES"."EMP NAME", 1,2)+'zzz' "EMPNAME" , "EMPLOYEES" . "SAL" from EMPLOYEES ; select * from customers where country = 'Mexico' DECLARE @ret nvarchar (15) ; EXEC @ret = dbo . ufnGetInventoryStock @EMPID = 201	Transact SQL commands are ignored for masking; however, statements are masked.

Use Case #2: SQL blocks that contain the USE <Database> command

Use case #2 provides an example of a scenario with SQL blocks that contain the USE <Database> command.

Example statement:

```
select * from employee
use ddm
select * from employee
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N or Y	EXECUTE GetTaskEvents2 'ALFKI' , 12209 select "EMPLOYEES" . "EMPID" , substring("EMPLOYEES"." EMPNAM E",1,2)+'zzz' "EM PNAME" , "EMPLOYEES" . "SAL" from EMPLOYEES ; select * from customers where country = 'Mexico' DECLARE @ret nvarchar (15) ; EXEC @ret = dbo . ufnGetInventoryStock @EMPID = 201 select employee . "EMPLOYEE_ID" , substring("EMPLOYEE"."L AST_NA ME",1,2)+'zzz' "LA ST_NAME" ,substring("EM PLOYEE"."FIRST_NA ME", 1,2)+'zzz' "FIRST_NAME" ,s ubstring("EMPLOYEE"."MI DDLE_N AME",1,2)+'zzz' "M IDDLE_NAME" ,employee . "JOB_ID" , employee . "MANAGER_ID" , employee . "HIREDATE" , employee . "SALARY" , employee . "COMM" , employee . "DEPARTMENT_ID" from employee use ddm select employee . "EMPID" ,substring("EMP LOYEE"."EMPNAME ", 1,2)+'zzz' "EMPNAME" , employee . "SAL" from employee	EXECUTE GetTaskEvents2 'ALFKI' , 12209 select "EMPLOYEES" . "EMPID" ,substring("EMP LOYEES"."EMP NAME", 1,2)+'zzz' "EMPNAME" , "EMPLOYEES" . "SAL" from EMPLOYEES ; select * from customers where country = 'Mexico' DECLARE @ret nvarchar (15) ; EXEC @ret = dbo . ufnGetInventoryStock @EMPID = 201select employee . "EMPLOYEE_ID" ,substrin g("EMPLOYEE"."LAST_ NAME",1,2)+'zzz' "LAST_N AME" , substring("EMPLOYEE"."F IRST _NAME",1,2)+'zzz' "FI RST_NAME" ,substring("E MPLOYEE"."MIDDL E_NAME",1,2)+'zzz' "MIDDL E_NAME" , employee . "JOB_ID" , employee . "MANAGER_ID" , employee . "HIREDATE" , employee . "SALARY" , employee . "COMM", employee . "DEPARTMENT_ID" from employee use ddm select employee . "EMPID" , substring("EMPLOYEE"."E MPN AME",1,2)+'zzz' "EMP NAME" ,employee . "SAL" from employee	Original Number of Rows check box is not relevant.

Use Case #3: SQL blocks that contain the SET QUOTED IDENTIFIER command

Use case #3 contains an example of a scenario with SQL blocks that contain the SET QUOTED IDENTIFIER command.

Example statement:

```
select * from employee
```

```
set quoted_identifier off
select * from employee
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N or Y	select employee . "EMPID" , substring("EMPLOYEE"."EMPNAME ",1,2)+'zzz' "EMPNAME" , employee . "SAL" from employee set quoted_identifier off select employee . EMPID , substring(employee . EMPNAME ,1,2)+'zzz' EMPNAME , employee . SAL from employee	select employee . "EMPID" , substring("EMPLOYEE"."EMPN AME",1,2)+'zzz' "EMPNAME" , employee . "SAL" from employee set quoted_identifier off select employee . EMPID , substring(employee . EMPNAME , 1,2)+'zzz'EMPNAME , employee . SAL from employee	Original Number of Rows check box is not relevant.

Use Case #4: SQL blocks that contain the EXECUTE AS command

Use case #4 contains an example of a scenario with SQL blocks that contain the EXECUTE AS command.

Example statement:

```
select * from employee
use zdb
execute as user = 'U21'
select * from emp
revert
execute as user = 'U200'
select * from emp1
revert
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N or Y	select employee . "EMPLOYEE_ID" , substring("EMPLOYEE"."LAST_NA ME",1,2)+'zzz' "LAST_NAME" , substring("EMPLOYEE"."FIRST_NA ME",1,2)+'zzz' "FIRST_NAME" , substring("EMPLOYEE"." MIDDLE_N AME", 1,2)+'zzz' "MIDDLE_NAME" , employee . "JOB_ID" , employee . "MANAGER_ID" , employee . "HIREDATE" , employee . "SALARY" , employee . "COMM" , employee . "DEPARTMENT_ID" from employee use zdb execute as user = 'U21' select emp . "EMPNO" , substring(emp . "ENAME", 1,2)+'zzz' "ENAME" , emp . "JOB" ,emp . "MGR" , emp . "HIREDATE" , emp. "SAL" , emp . "COMM" , emp . "DEPTNO" from emp revert execute as user = 'U200' select emp1 . "CEID" , substring(emp1 . "CNAME" ,1,2) + 'zzz' "CNAME" from emp1 revert	select employee . "EMPLOYEE_ID" , substring("EMPLOYEE"."LAST_ NAME",1,2)+'zzz' "LAST_NAME" , substring("EMPLOYEE"."FIRST_NAME",1,2)+'zzz' "FIRST_NAME" , substring("EMPLOYEE"." MIDDL E_NAME", 1,2)+'zzz'"MIDDLE_NAME" , employee . "JOB_ID" , employee . "MANAGER_ID" , employee . "HIREDATE" , employee . "SALARY" , employee . "COMM" , employee . "DEPARTMENT_ID" fromemployee use zdb execute as user = 'U21' select emp . "EMPNO" , substring(emp . "ENAME" ,1,2)+'zzz' "ENAME" , emp . "JOB" , emp . "MGR" , emp . "HIREDATE" , emp. "SAL" , emp . "COMM" , emp . "DEPTNO" from emp revert execute as user = 'U200' select emp1 . "CEID" , substring(emp1 . "CNAME" ,1,2) + 'zzz' "CNAME" from emp1 revert	Original Number of Rows check box is not relevant.

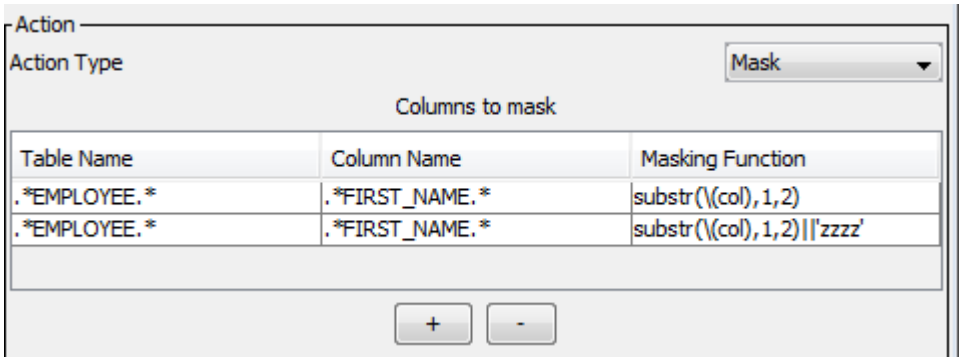
Use Cases Specific to Security Rule Design

This section describes use cases specific to security rule design.

Security Rule Design Use Case #1

Use case #1 provides an example of a scenario where a security rule has two rows specifying the same table and column for masking, but different masking functions.

The following image shows an example:



Example statement:

```
SELECT DEPARTMENT_ID, first_name, salary
```

The following table describes the example masking scenarios:

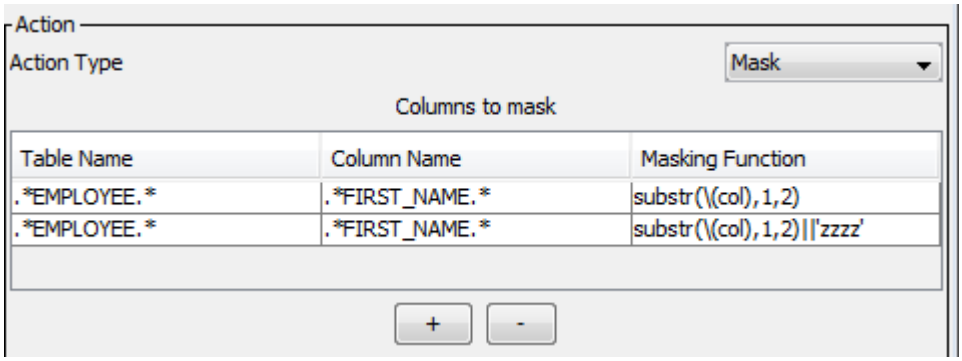
Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N or Y	SELECT DEPARTMENT_ID , substr("FIRST_NAME", 1,2)||'zzzz' "FIRST_NAME" , salary FROM EMPLOYEES	SELECT DEPARTMENT_ID , substr(first_name ,1,2) first_name , salary FROM EMPLOYEES	The candidate column is masked only by using one of the masking rules defined in the masking action. Old masking functionality: When enabled, always applies the last corresponding masking function from the set of rows in the security rule for the same table and column name. New masking functionality: When enabled, always applies the first corresponding masking function from the set of rows in the security rule for the same table and column name.

Security Rule Design Use Case #2

Use case #2 provides an example of the concatenation of column masking by applying several consequent security rules on the same column.

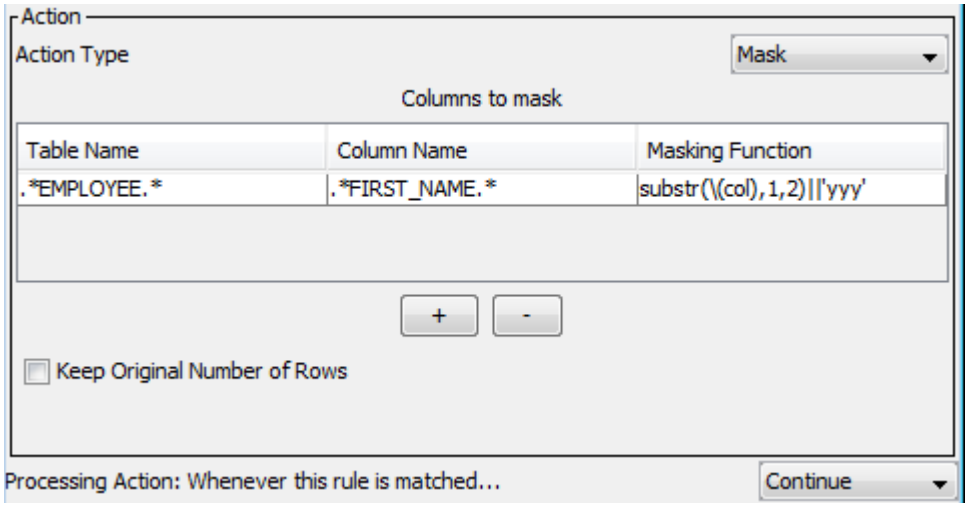
First Security Rule

The following image shows an example of the first security rule:



Second Security Rule

The following image shows an example of the second security rule:



Example statement:

```
Select First_name, last_name, DEPARTMENT_ID, salary from EMPLOYEES
```

The following table describes the example masking scenarios:

Keep Original Number of Rows	Old Masking Statement Modification	New Masking Statement Modification	Masking Action Functionality
N or Y	Select substr (substr("FIRST_NAME", 1,2)||'yyy' , 1 , 2) || 'zzzz' "FIRST_NAME" , LAST_NAME , DEPARTMENT_ID ,salary from EMPLOYEES	select substr (substr(first_name , 1,2)||'yyy' , 1 , 2) first_name , last_name , DEPARTMENT_ID , salary from EMPLOYEES	The security rules are applied consequently on the candidate column in the original query.

Authors

Vamsi Krishna Brahmadevara

Arpit Garg

Yuri Gauchman

Jennifer Osburn

Madhavi Darapu